

# REPORT HOMEWORK 2 CMRM2024

Francesco Crociani and Luca Trapella

January 2025



# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                                    | <b>5</b>  |
| <b>1 Loading and processing audio files</b>                            | <b>7</b>  |
| 1.1 STFT . . . . .                                                     | 8         |
| 1.2 Time and Frequency Resolution . . . . .                            | 10        |
| <b>2 Nonnegative Matrix Factorization</b>                              | <b>11</b> |
| 2.1 Initializing W and H . . . . .                                     | 11        |
| 2.2 NMF parameters . . . . .                                           | 12        |
| 2.3 Frobenius Norm Error . . . . .                                     | 13        |
| 2.4 Visualizing the effects of NMF . . . . .                           | 14        |
| <b>3 Timbre Transfer</b>                                               | <b>23</b> |
| 3.1 Inverse STFT and Griffin-Lim algorithm . . . . .                   | 23        |
| 3.2 Phase Visualization . . . . .                                      | 25        |
| <b>4 Timbre Transfer Funtion</b>                                       | <b>29</b> |
| <b>5 Final Results</b>                                                 | <b>31</b> |
| 5.1 Timbre transfer for all combinations . . . . .                     | 31        |
| 5.2 Experimenting with parameters . . . . .                            | 32        |
| 5.3 NMF parameters . . . . .                                           | 34        |
| 5.4 Additional processing steps to improve timbre transfer results . . | 35        |
| 5.4.1 Checking timbre similarities . . . . .                           | 36        |



# Introduction

The focus of this report is the application of timbre transfer. The aim is to implement an algorithm that is able to transfer the timbre of a particular source to a target audio signal relying only on signal processing methods, in particular on **Nonnegative Matrix Factorization** (NMF) and evaluating the results.



# Chapter 1

## Loading and processing audio files

The first thing we do is loading the audio files from the two subfolders related to sources and targets. The audio/sources folder contains three sources representing background sounds from which we are going to learn the timbre, namely *Bees Buzzing.wav*, *Wind Blowing.wav*, and *Chainsaw Sawing.wav*. The audio/targets folder contains four musical tracks targets to which we are going to apply the learned timbre, namely *Jingle Bells Boogie.wav*, *Have Yourself.wav*, *Blue Christmas.wav*, and *White Christmas.wav*. The goal of this section is to prepare the audio signals to ensure they can be processed in a consistent manner during the analysis phase.

To begin with, we define the paths to the directories where the source and target audio files are stored. The paths for the source and target audio files are specified, and the names of the source and target files are selected. In the first part of the problem, we are going to work only with *Bees Buzzing* as source and *Jingle Bells Boogie* as target to begin to understand the task and develop a pipeline. The next step involves setting the sampling frequency to 22050 Hz for the audio files and loading them. The *librosa.load()* function is used to load both the source and target audio files. The following code loads the source audio file ("Bees\_Buzzing.mp3") and the target audio file ("Jingle\_Bells\_Boogie.wav"):

```
1 source_file = f"{source_dir}/{sources[0]}.mp3"  
2 target_file = f"{target_dir}/{targets[2]}.wav"  
3 source_waveform, source_sr = librosa.load(source_file,  
    sr=Fs) # Librosa loads the file audio mono as default  
4 target_waveform, target_sr = librosa.load(target_file,  
    sr=Fs) # Librosa loads the file audio mono as default
```

The `sr=Fs` argument ensures that both audio files are loaded at the desired sampling rate of 22050 Hz, regardless of the original sampling rate of the files. Additionally, by default, Librosa converts stereo files to mono when loading them.

## 1.1 STFT

We then compute the **Short-Time Fourier Transform** (STFT) to analyze the frequency content of the audio signals over time. The STFT is widely used in signal processing to transform a time-domain signal into a time-frequency representation, which is essential for visualizing and understanding the spectral characteristics of the audio. This transformation allows us to observe how the frequency content evolves over time for both the source and target signals.

To compute the STFT, two key parameters are defined:

- **Window size** (N\_length): The length of the analysis window is set to 4096 samples. This determines the frequency resolution.
- **Hop size** (H\_size): The hop length, which determines the overlap between adjacent windows, is set to 1024 samples. This parameter affects the time resolution of the analysis.

We compute the STFT for both the source and target signal as follows:

```

1  N_length = 4096
2  H_size = 1024
3
4  # STFT computation
5  # Source
6  X_source = librosa.stft(source_waveform, n_fft=N_length,
7                           hop_length=H_size)
8  Y_source = np.abs(X_source)
9
10 # Plot
11 plt.figure(figsize=(10, 6))
12 librosa.display.specshow(librosa.amplitude_to_db(Y_source,
13                                                    ref=np.max), sr=Fs, hop_length=H_size, x_axis='time',
14                           y_axis='log')
15 plt.colorbar(format="%+2.0f dB")
16 plt.title(f"Spectrogram (Magnitude) of Source:
17           {sources[0]}")
18
19 plt.xlabel("Time (s)")
20 plt.ylabel("Frequency (Hz)")
21 plt.show()
22
23 # Target
24 X_target = librosa.stft(target_waveform, n_fft=N_length,
25                           hop_length=H_size)
26 Y_target = np.abs(X_target)
27
28 # Plot
29 plt.figure(figsize=(10, 6))

```



```

25 librosa.display.specshow(librosa.amplitude_to_db(Y_target,
    ref=np.max), sr=Fs, hop_length=H_size, x_axis='time',
    y_axis='log')
26 plt.colorbar(format="+2.0f dB")
27 plt.title(f"Spectrogram (Magnitude) of Target:
    {targets[2]}")
28 plt.xlabel("Time (s)")
29 plt.ylabel("Frequency (Hz)")
30 plt.show()

```

The Spectrograms are also evaluated.

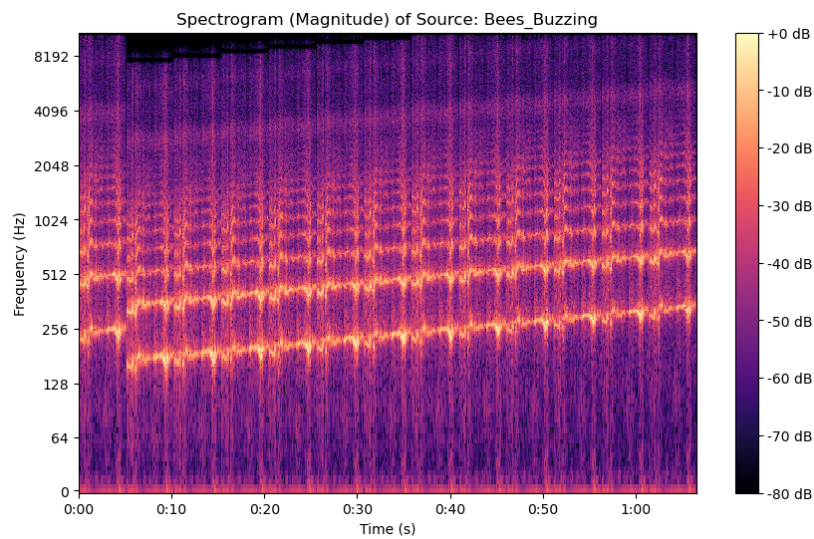


Figure 1.1: Bees\_Buzzing spectrogram

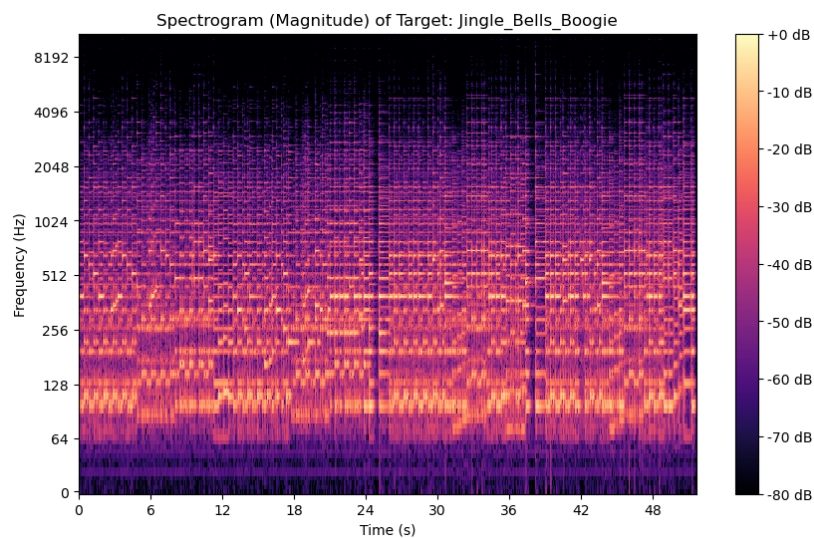


Figure 1.2: Jingle\_Bells\_Boogie spectrogram

We notice some differences between the beezing sound and jingle bells spectrograms. The Bees Buzzing has a simpler, repetitive pattern with concentrated energy and stability in time and frequency. Jingle Bells has a more complex, irregular and dynamic spectrogram with a significant time variation. These differences reflect the contrast between a natural, monotonic sound and a structured, harmonic musical composition.

## 1.2 Time and Frequency Resolution

The concepts of time resolution and frequency resolution are critical in signal processing, particularly when analyzing signals using the Short-Time Fourier Transform.

- **Time Resolution** ( $time\_res = H\_size / Fs = 0.0464$  seconds) refers to the minimum duration of time that can be resolved or distinguished in the analysis.  $H\_size$  is the hop size (step size of the window in time) and  $Fs$  is the sampling frequency in Hz. This formula implies that the time resolution depends on how much the signal advances (in seconds) during one step of the analysis window.
- **Frequency Resolution** ( $freq\_res = Fs / N\_length = 5.38$  Hz) indicates the smallest frequency difference that can be resolved in the analysis. It depends on the sampling frequency and the **FFT** length ( $N\_length$ ). This formula suggests that higher  $N\_length$  improves frequency resolution by adding finer granularity in the frequency domain.

Time and frequency resolutions are inversely related. Increasing the window length  $H\_size$  improves frequency resolution but worsens time resolution. Conversely, decreasing the  $H\_size$  improves the resolution of time but sacrifices the resolution of frequencies.

## Chapter 2

# Nonnegative Matrix Factorization

Now that we have loaded the audio files, we are going to accomplish timbre transfer by means of NMF. Nowadays, timbre transfer is accomplished by making use of very large and sophisticated neural networks, but standard signal processing techniques can be used to obtain a "quick and dirty" solution, and NMF is among them. NMF is a technique used to approximate a non-negative matrix  $\mathbf{V}$  as a product of two non-negative matrices  $\mathbf{W}$  and  $\mathbf{H}$ :

- $\mathbf{W}$ : Represents the spectral templates or basis components.
- $\mathbf{H}$ : Represents the temporal activations or weights of the components.

The matrix  $\mathbf{V}$  represents a magnitude spectrum of a signal with  $K$  frequency bins (rows) and  $N$  time frames (columns). All values are nonnegative since they represent magnitudes or amplitudes of frequency components. By initializing the templates  $\mathbf{W}$  using the source spectrogram and iteratively optimizing the activations  $\mathbf{H}$ , the NMF aims to approximate the target spectrogram. The Kullback-Leibler Divergence (KLDiv) is used as the cost function to minimize the difference between the target spectrogram and its NMF-based approximation.

### 2.1 Initializing $\mathbf{W}$ and $\mathbf{H}$

Before performing the NMF, the spectral templates  $\mathbf{W}$  and activations  $\mathbf{H}$  are initialized as follows:

```
1 # Initialize activations randomly
2 H0 = np.random.rand(X_source.shape[1], X_target.shape[1])
3
4 # Initialize templates according to source frames
5 epsilon = 1e-6
6 W0 = Y_source / (np.sum(Y_source, axis=0, keepdims=True) +
    epsilon)
```

```

7  #define the variable Xs which is the normalized version of
   X_source
8  Xs = X_source / (np.sum(Y_source, axis=0, keepdims=True) +
   epsilon)

```

The activations  $H$  are initialized randomly with dimensions corresponding to the time frames of the source and target spectrograms.

The spectral templates  $W$  are initialized using the source magnitude spectrogram  $Y_{\text{source}}$ . Each column of  $W_0$  is normalized to ensure proper scaling. The source STFT is normalized in a similar manner for consistency. The normalization ensures numerical stability by preventing divisions by zero (controlled via  $\epsilon$ ) and ensuring proper scaling for the iterative NMF updates.

## 2.2 NMF parameters

The NMF algorithm then is applied to decompose the target spectrogram  $Y_{\text{target}}$  using the initialized templates  $W_0$  and activations  $H_0$ . The key parameters for the NMF are:

- **Cost function:** Kullback-Leibler Divergence (KLDiv), which measures the difference between the true and approximated spectrograms.
- **Number of Iterations:** 50, to allow convergence of the optimization.
- **Template fixity:**  $W$  is fixed during the optimization process ( only  $H$  is updated).
- **Polyphony control:** set to 10, controls continuity and sparsity in the activations for smoother results.

The following code applies the NMF:

```

1  # NMF
2  W,H= nmf(
3      V=Y_target,
4      cost_func='KLDiv', # Using Kullback-Leibler Divergence
5      num_iter=50, # Set number of iterations
6      init_W=W0, # Initialize W with W0
7      init_H=H0, # Initialize H with H0
8      fix_W=True, # Fix W during updates
9      cont_polyphony=10, # Control polyphony continuity
10     cont_length=7, # Length of activated templates for
        continuity
11     cont_grid=5, # Frequency at which continuity is
        enforced
12     cont_sparsen=(1, 7), # Sparsity control
13 )

```

After completing the NMF process, the approximate magnitude spectrogram  $Y_{\text{approx}}$  is reconstructed by performing matrix multiplication between  $W$  and  $H$ :

```
1 Y_approx = W @ H # @ = matrix multiplication in python
```

This approximation represents the target spectrogram in terms of the spectral components derived from the source spectrogram.

## 2.3 Frobenius Norm Error

To evaluate the quality of the approximation, the Frobenius norm of the error between the true and approximated magnitude spectrograms is computed:

$$Error = ||Y_{\text{target}} - Y_{\text{approx}}||_F \quad (2.1)$$

The Frobenius norm measures the overall magnitude of the difference between the two matrices.

```
1 error = np.linalg.norm(Y_target - Y_approx, 'fro') #  
    Frobenius norm applied to a 2D array
```

The output provides a quantitative measure of the approximation quality. In our case an error of 5003 is evaluated. This value is relatively high. This suggests that the approximation is not very close to the original magnitude STFT ( $Y_{\text{target}}$ ). In fact, a large error means that the model hasn't learned the target magnitude spectrum well. Some possible reasons for such a value could be:

- **Initialization:** The initialization of  $W$  and  $H$  plays a significant role in the final approximation. If the initial matrices are far from the optimal solution, the NMF may struggle to converge to a good approximation (in our case  $H_0$ ).
- **NMF Parameters:** The parameters for the NMF algorithm (e.g., number of iterations, sparsity control, polyphony constraints) may not be optimal. If these parameters aren't set correctly, they can hinder the algorithm's ability to capture the underlying structure of the target.
- **Model Constraints:** The parameters like `fix_W=True`, `cont_polyphony`, `cont_length`, and others might limit the flexibility of the model. If these constraints are too rigid, the model may not have enough freedom to capture the complexities of the target signal.
- **Cost Function:** The choice of cost function ( `KLDiv` in this case) can affect the quality of the approximation. KL divergence is sensitive to the initial values and can sometimes result in a higher error if the model is not well-tuned.

## 2.4 Visualizing the effects of NMF

Now, the activation matrices  $H_0$  (initialized activations) and  $H$  (updated activations after NMF) are visualized to analyze the changes introduced by the Non-negative Matrix Factorization (NMF) process. Additionally, a logarithmic post-processing step is applied to the updated activation matrix  $H$  to enhance and highlight the activations. The activation matrix  $H$  represents how much each template (basis function) contributes to the magnitude of the original spectrogram at different time frames. Each column corresponds to a time frame in the spectrogram, and each row corresponds to a different template. Larger values in  $H$  indicate higher activity or presence of a certain spectral feature at that time.

```
1  # Before NMF - Initial H0
2  plt.figure(figsize=(12, 6))
3  plt.imshow(H0, aspect='auto', origin='lower',
4             cmap='viridis', interpolation='nearest')
5  plt.colorbar(label='Activation Value')
6  plt.title('Activation Matrix H0 (Before NMF)')
7  plt.xlabel('Time Frames')
8  plt.ylabel('Activations')
9  plt.show()
10
11 # After NMF - Updated H
12 plt.figure(figsize=(12, 6))
13 plt.imshow(H, aspect='auto', origin='lower',
14           cmap='viridis', interpolation='nearest')
15 plt.colorbar(label='Activation Value')
16 plt.title('Activation Matrix H (After NMF)')
17 plt.xlabel('Time Frames')
18 plt.ylabel('Activations')
19 plt.show()
```



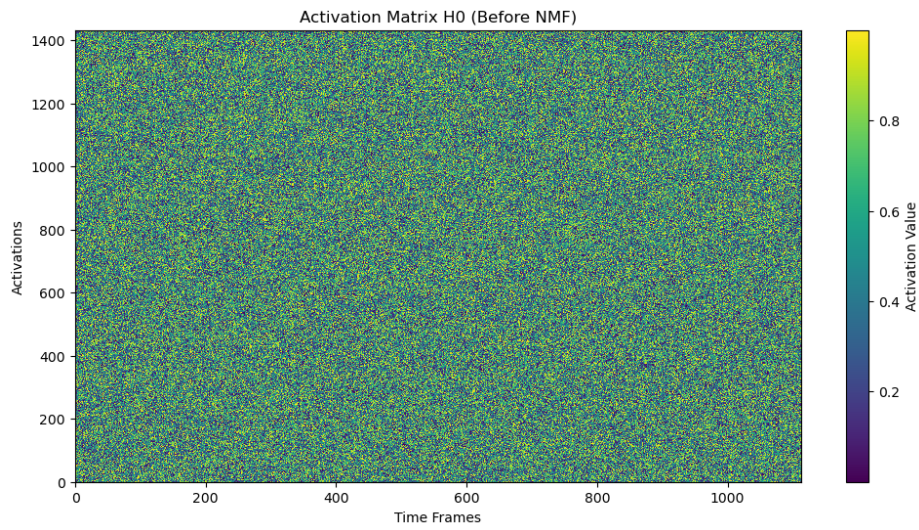


Figure 2.1: Activation Matrix H0 before NMF

This plot shows the initial activations of the templates before applying NMF. It's noisy and have low values since these activations are initialized randomly. It can provide an idea of how the templates are initialized and how sparsely they are distributed across the time frames.

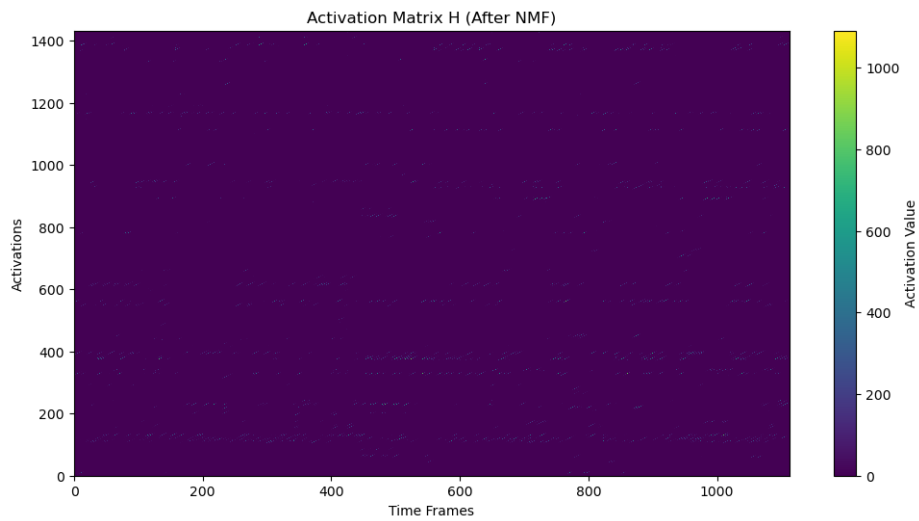


Figure 2.2: Activation Matrix H after NMF

After NMF, this plot shows the learned activations of the templates. The matrix H represents how much each template is active over time.

To further analyze the activation matrix  $H$ , a logarithmic transformation is applied. This transformation enhances the visibility of small activations while compressing the range of larger values, highlighting smaller activations that may otherwise remain hidden and providing a clearer representation of temporal activation patterns.

$$H_{processed} = \log(1 + \gamma |H|) \quad (2.2)$$

```

1  # Post-processing on H to highlight activations
2  gamma = 2
3  H_processed = np.log(1+gamma*np.abs(H))
4
5  # Plot post-processed H
6  plt.figure(figsize=(12, 6))
7  plt.imshow(H_processed, aspect='auto', origin='lower',
8             cmap='viridis', interpolation='nearest')
9  plt.colorbar(label='Activation Value')
10 plt.title('Post-processed Activation Matrix H')
11 plt.xlabel('Time Frames')
12 plt.ylabel('Activations')
13 plt.show()

```

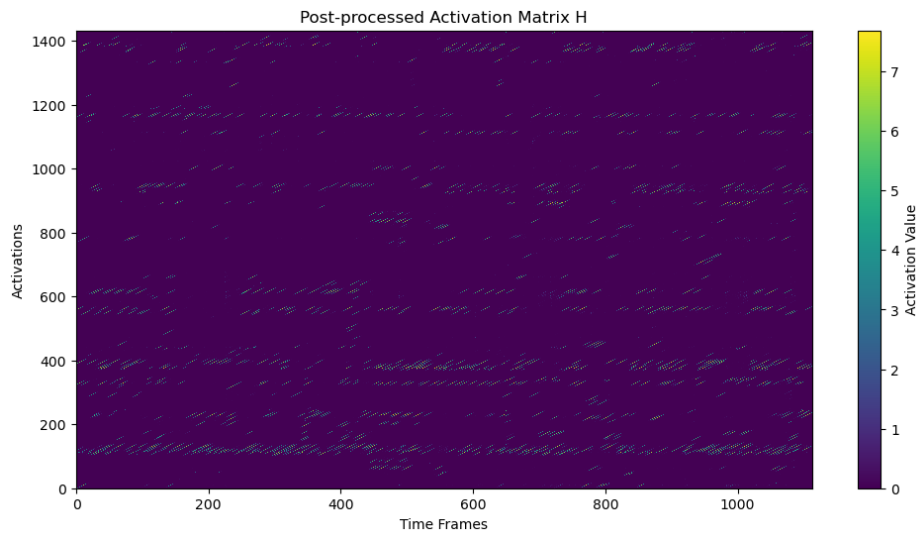


Figure 2.3: Post-processed activation matrix  $H$

We then proceed to define the function `visualize_nmf` which takes in the magnitude STFT (in our case of  $Y$  target), the activation matrix, the template matrix, the time resolution and the compression factor  $\gamma$  and plots a compressed version of  $H$ ,  $W$  and of the approximation obtained using  $W$  and  $H$ . It also plots a compressed version of the magnitude STFT of  $Y$  target.



```

1  def visualize_nmf(V, W, H, fs, time_res, gamma=2):
2      """Visualize the matrices W, H, and compare
3          reconstruction  $V \sim WH$  to V
4
5      Args:
6          V: target
7          W: template matrix
8          H: activation matrix
9          fs: sampling frequency
10         time_res: time resolution
11         gamma: compression factor for V
12
13     Returns:
14         None
15     """
16     # 1. Plot compressed version of the activation matrix H
17     H_compressed = np.log(1+gamma*np.abs(H)) # Apply
18         compression
19     plt.figure(figsize=(12, 6))
20     plt.imshow(H_compressed, aspect='auto',
21         origin='lower', cmap='viridis',
22         interpolation='nearest')
23     plt.colorbar()
24     plt.title(f'Compressed Activation Matrix H with gamma
25         {gamma}')
26     plt.xlabel('Time frames')
27     plt.ylabel('Activations')
28     plt.show()
29
30     # 2. Plot compressed version of the template matrix W
31     W_compressed = np.log(1+gamma*np.abs(W)) # Apply
32         compression
33     plt.figure(figsize=(12, 6))
34     plt.imshow(W_compressed, aspect='auto',
35         origin='lower', cmap='viridis',
36         interpolation='nearest')
37     plt.colorbar()
38     plt.title(f'Compressed Template Matrix W with gamma
39         {gamma}')
40     plt.xlabel('Templates')
41     plt.ylabel('Frequency bins')
42     plt.ylim(0, 2000) # constrain y-axis between 0 and
43         2000 Hz
44     plt.show()
45
46     # 3. Plot compressed version of the approximation ( $V \sim
47         WH$ )

```

```

37     V_approx = W @ H
38     V_approx_compressed = np.log(1+gamma*np.abs(V_approx))
39     # Apply compression
40     plt.figure(figsize=(12, 6))
41     plt.imshow(V_approx_compressed, aspect='auto',
42               origin='lower', cmap='viridis',
43               interpolation='nearest')
44     plt.colorbar()
45     plt.title(f'Compressed Approximation of V (WH) with
46             gamma {gamma}')
47     plt.xlabel('Time frames')
48     plt.ylabel('Frequency bins')
49     plt.ylim(0, 2000) # constrain y-axis between 0 and
50                       2000 Hz
51     plt.show()
52
53     # 4. Plot compressed version of the magnitude STFT of
54     # Y target
55     V_compressed = np.log(1+gamma*np.abs(V)) # Apply
56     # compression using a percentile range of [0, 100]
57     plt.figure(figsize=(12, 6))
58     plt.imshow(V_compressed, aspect='auto',
59               origin='lower', cmap='viridis',
60               interpolation='nearest')
61     plt.colorbar()
62     plt.title(f'Compressed Magnitude STFT of Target
63             (Y_target) with gamma {gamma}')
64     plt.xlabel('Time frames')
65     plt.ylabel('Frequency bins')
66     plt.ylim(0, 2000) # constrain y-axis between 0 and
67                       2000 Hz
68     plt.show()

```

We are also going to test different  $\gamma$  values, visualizing in the report the case for  $\gamma=2$ .

```

1 visualize_nmf(Y_target, W, H, fs=Fs, time_res=time_res,
2             gamma=1) # gamma = 1
3 visualize_nmf(Y_target, W, H, fs=Fs, time_res=time_res,
4             gamma=2) # gamma = 2
5 visualize_nmf(Y_target, W, H, fs=Fs, time_res=time_res,
6             gamma=4) # gamma = 4

```

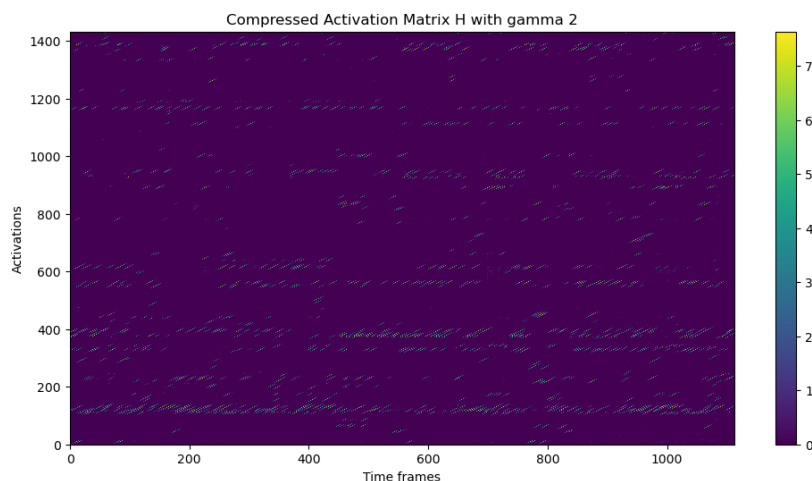


Figure 2.4: Compressed activation matrix H

The activation matrix  $H$  is clipped using the compression factor  $\gamma$ . The `np.percentile` function helps to compress the values to a certain range. This visualization shows how much each template is active across different time frames after the NMF process.

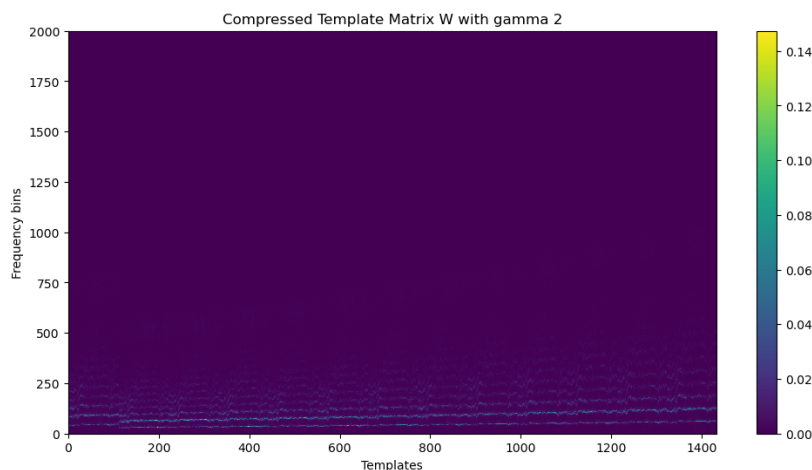
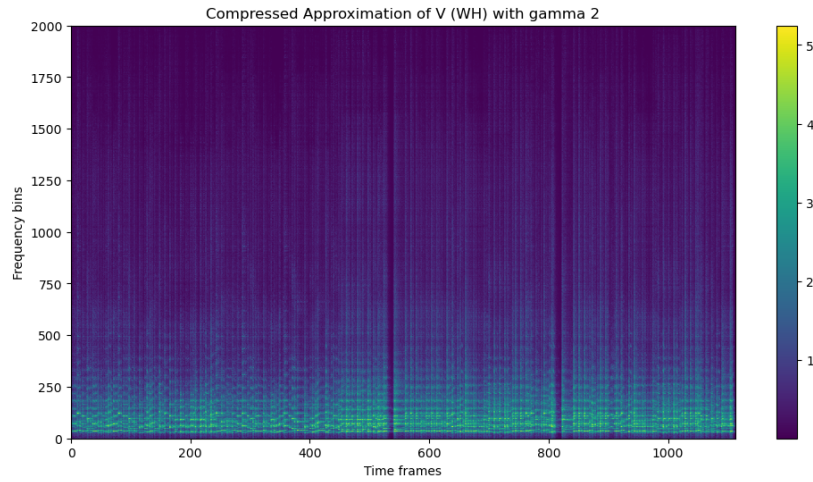


Figure 2.5: Compressed template matrix W

The template matrix  $W$  is also clipped based on the compression factor  $\gamma$ . This matrix represents the spectral templates learned by NMF. We also limit the y-axis to 2000 Hz to focus on the relevant frequency range.

Figure 2.6: Compressed approximation of  $V$ 

The magnitude spectrogram  $V_{\text{approx}}$  is computed as  $W @ H$ . This approximation is then clipped and displayed, showing how well the NMF model has approximated the target spectrogram.

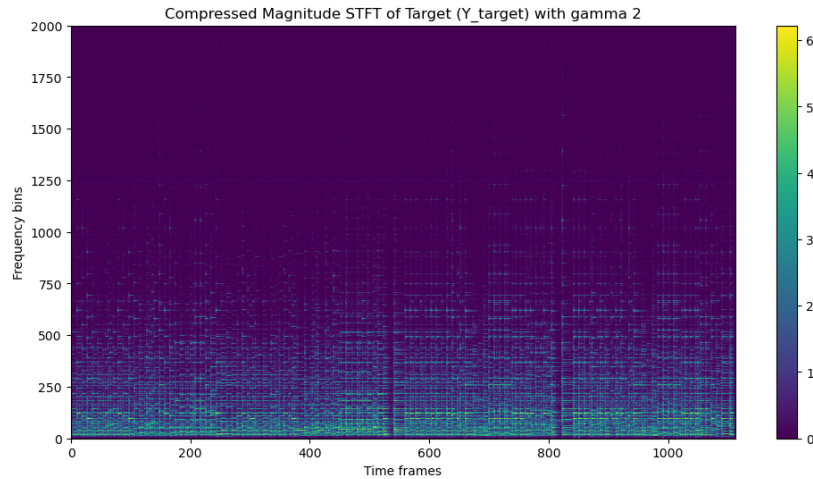


Figure 2.7: Compressed Magnitude STFT

Finally, the target spectrogram  $V$  ( $Y_{\text{target}}$ ) is clipped and displayed, allowing you to compare it to the approximation generated by  $W$  and  $H$ .

The overall structure of both representations is similar, with stronger intensities in the lower frequencies (green hues in the lower part) and a gradual decay of energy at higher frequencies. However, the approximation appears less detailed compared to the original target STFT. This is particularly evident in the higher frequency regions, where energy is more blurred and less defined.

- **Loss of Detail:** The approximation simplifies spectral details since NMF approximates the signal using a finite number of templates (columns of  $W$ )

and activations ( $H$ ). This results in a more "generalized" representation compared to the original STFT.

- **Loss of Temporal and Spectral Variations:** The approximation loses some high-frequency components and minor temporal variations, especially those of lower intensity, which are still visible in the original target  $Y_{\text{target}}$

The most critical parameters influencing the NMF factorization are the cost function, the initialization of  $W$  and  $H$ , and the choice to fix  $W$  while learning  $H$ . The plots comparing the original magnitude STFT  $V$  and its approximation  $WH$  reveal that while the approximation captures broad patterns, discrepancies arise when spectral components in  $V$  are not present in the fixed  $W$ . Fixing  $W$  simplifies the factorization process and enforces the spectral structure of the source signal while allowing  $H$  to adapt temporally to approximate the target. However, this constraint limits the model's ability to adapt to unique features in the target signal. By adjusting parameters such as the number of iterations, sparsity, and continuity constraints, the factorization quality can be further improved.



# Chapter 3

## Timbre Transfer

Once the activation matrix  $H$  has been learned, it can be utilized for timbre transfer. Specifically,  $H$  is applied to the complex-domain STFT (Short-Time Fourier Transform), which includes both magnitude and phase information. This approach ensures that the phase is not completely discarded during the transformation process.

### 3.1 Inverse STFT and Griffin-Lim algorithm

To reconstruct the time-domain waveform from this representation, two alternative methods can be employed:

1. **Inverse STFT (ISTFT):** A straightforward approach that uses the existing phase information in the transformed data.
2. **Griffin-Lim Algorithm:** A more sophisticated iterative method designed to handle situations where the phase information is incomplete or entirely missing.

It is important to note that reconstructing the audio is not a trivial task because the correct phase for the processed audio is typically unknown. This is where the Griffin-Lim algorithm becomes particularly valuable.

The Griffin-Lim algorithm is an iterative signal processing technique used to reconstruct a time-domain signal when only the magnitude spectrogram is available. It works by iteratively refining the phase while keeping the magnitude fixed, thereby minimizing the reconstruction error at each step. Although it does not guarantee a perfect reconstruction of the original signal, it is highly effective in practical scenarios. This makes it especially popular in applications such as speech synthesis and audio processing, where retaining perceptual quality and timbre is critical despite missing or approximated phase information.

The first thing we do is replacing magnitude frames by complex-valued frames.

```
1 # Replace the magnitude frames by complex valued frames
2 Y_tt = Xs @ H
```

The learned activation matrix is applied to the template matrix to generate the magnitude spectrogram. This step provides a representation of the audio signal in the complex domain. However, the exact phase information is not available, so an approximate phase was implicitly used in the subsequent steps.

We then re-synthesize the audio by means of the Inverse STFT.

```

1 # Re-synthesize using ISTFT
2 y_istft = librosa.istft(Y_tt, hop_length=H_size) # has the
   same phase of Ytt but is not the original one

```

The reconstructed waveform was plotted over time to observe the amplitude variations.

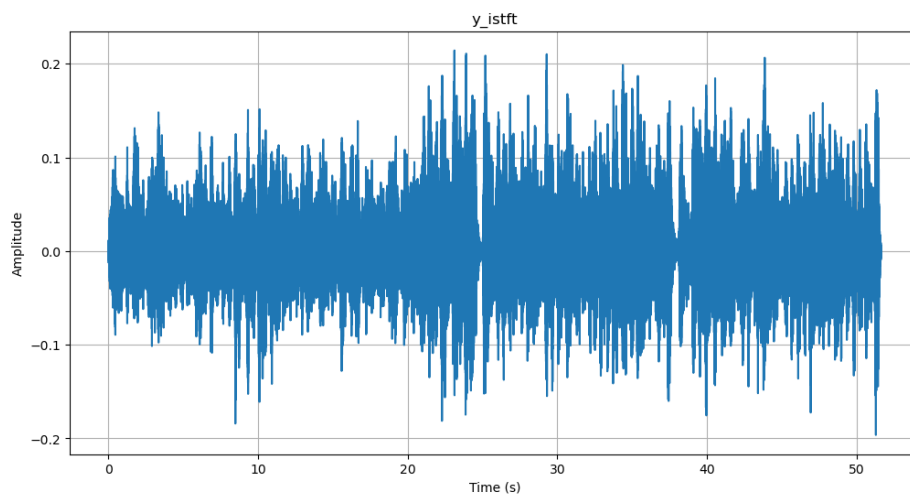


Figure 3.1: ISTFT reconstruction

This method is computationally efficient but may result in suboptimal reconstruction due to the use of approximate phase information.

We do the same for the Griffin-Lim algorithm.

```

1 # Re-synthesize using Griffin-Lim algorithm
2 y_gl = librosa.griffinlim(np.abs(Y_tt), hop_length=H_size,
   win_length=N_length)

```

The Griffin-Lim algorithm was employed to iteratively refine the phase while keeping the magnitude fixed. This algorithm is specifically designed to handle scenarios where the exact phase information is missing, iteratively estimating the phase to minimize reconstruction error. Each execution of the algorithm can yield slightly different results due to its iterative nature.

The reconstructed waveform was plotted, showing the time-domain signal obtained using the Griffin-Lim approach.



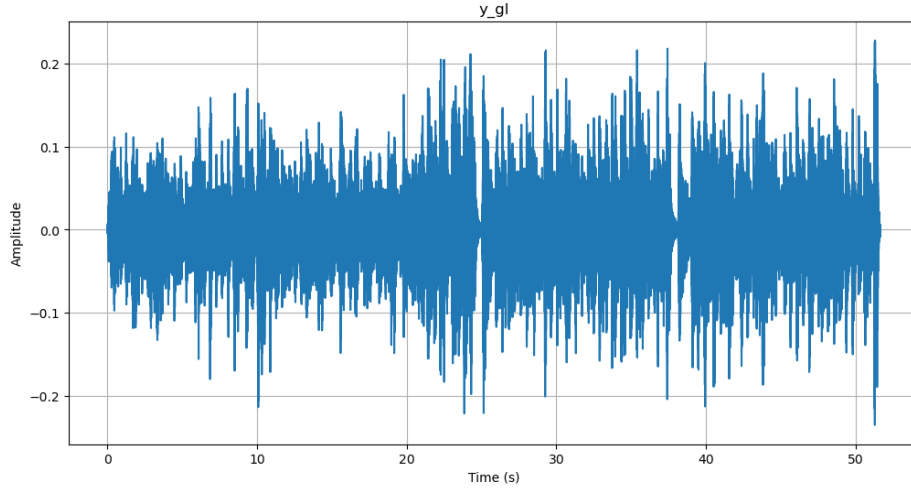


Figure 3.2: Griffin-Lim algorithm reconstruction

While computationally more intensive, the algorithm often produces more perceptually accurate results compared to ISTFT.

## 3.2 Phase Visualization

We proceed to compute the phase and plot the first ten seconds of some frequencies of interest:

- For the ISTFT result (`Y_tt`), the phase is directly extracted using the `np.angle()` function.

```
1 Y_istft_phase = np.angle(Y_tt)
```

- For the Griffin-Lim result (`y_gl`), the Short-Time Fourier Transform (STFT) is computed again to obtain its complex representation. The phase is then extracted similarly.

```
1 Y_gl_complex = librosa.stft(y_gl, hop_length=H_size,
    win_length=N_length, n_fft=n_fft)
2 Y_gl_phase = np.angle(Y_gl_complex)
```

The analysis focuses on the following frequencies: 250 Hz, 500 Hz, 1000 Hz, 1500 Hz, and 2000 Hz. The corresponding frequency bin indices are calculated using the formula:

$$bin\_index = round\left(\frac{frequency \cdot N\_length}{F_s}\right) \quad (3.1)$$

```
1 frequency_bins = [int(np.round(f * N_length / Fs)) for f
    in frequencies]
```

For each frequency, the phase is plotted over the time axis for both  $Y_{tt}$  (ISTFT) and  $Y_{gl}$  (Griffin-Lim) reconstructions.

```

1  # Number of frames for the first 10 seconds
2  num_frames = int(10 * Fs / H_size)
3  time_axis = np.linspace(0, 10, num_frames)
4
5  # Plot phase for each frequency for Y_tt and y_gl
6  plt.figure(figsize=(12, 15))
7
8  for i, f in enumerate(frequencies):
9      # Subplot for Y_tt phase at the specific frequency
10     plt.subplot(len(frequencies)*2, 1, 2*i + 1)
11     plt.plot(time_axis, np.angle(Y_tt[frequency_bins[i],
12     :num_frames]), label=f"Phase at {f} Hz")
13     plt.title(f"Phase of Y_tt at {f} Hz (First 10
14     seconds)")
15     plt.xlabel("Time (s)")
16     plt.ylabel("Phase (radians)")
17     plt.grid(True)
18     plt.legend()
19
20     # Subplot for y_gl phase at the specific frequency
21     plt.subplot(len(frequencies)*2, 1, 2*i + 2)
22     # Extract phase at the same frequency for y_gl
23     gl_phase = np.angle(Y_gl_complex[frequency_bins[i],
24     :num_frames])
25     plt.plot(time_axis, gl_phase, label=f"Phase at {f}
26     Hz", color='blue')
27     plt.title(f"Phase of y_gl at {f} Hz (First 10
28     seconds)")
29     plt.xlabel("Time (s)")
30     plt.ylabel("Phase (radians)")
31     plt.grid(True)
32     plt.legend()
33
34 # Adjust layout
35 plt.tight_layout()
36 plt.show()

```

The Griffin-Lim algorithm reconstructs the signal iteratively, and as a result, the phase is estimated without modifying the amplitude. The phase plot will often appear more uniform or exhibit distinct patterns compared to the phase of the  $y_{istft}$ .

We then listen to the results by employing the `ipd.Audio` function.

```
1 # Play target
2 print("Playing Target Audio:")
3 ipd.display(ipd.Audio(target_waveform, rate=target_sr))
4 # Play source
5 print("Playing Source Audio:")
6 ipd.display(ipd.Audio(source_waveform, rate=source_sr))
7 # Play new target - ISTFT
8 print("Listening to the ISTFT result:")
9 ipd.display(ipd.Audio(y_istft, rate=Fs))
10 # Play new target - GF
11 print("Listening to the Griffin-Lim result:")
12 ipd.display(ipd.Audio(y_gl, rate=Fs))
```

Phase estimation is not perfect, and these discrepancies might lead to audible differences between the reconstructed signals. Both methodologies introduce slight artifacts in the audio, creating perceptual differences. However, the Griffin-Lim method proves somewhat more effective at timbre transfer. From the audio we can hear that with this method the song is clearer and not too noisy like the one obtained with the `istft`. The quality of this transfer depends on several key factors:

- **Magnitude Preservation:** The accuracy of the magnitude STFT transfer (via NMF) directly impacts the timbre transfer. Matching the source and target magnitudes effectively is essential.
- **Phase Preservation:** Retaining the original phase (using ISTFT) or estimating phase (using Griffin-Lim) significantly affects the naturalness of the transferred timbre.
- **NMF Parameters:** Initialization of `H` and `W` influences how well the source timbre is encoded and transferred. Sparsity control impacts the balance between the timbre's richness and simplicity.
- **Window and Hop Size:** The resolution of the STFT affects how accurately the frequency content and temporal dynamics of the source are captured and transferred to the target.
- **Polyphony and Continuity Controls:** These parameters affect how well overlapping components and complex textures are handled in the timbre transfer.

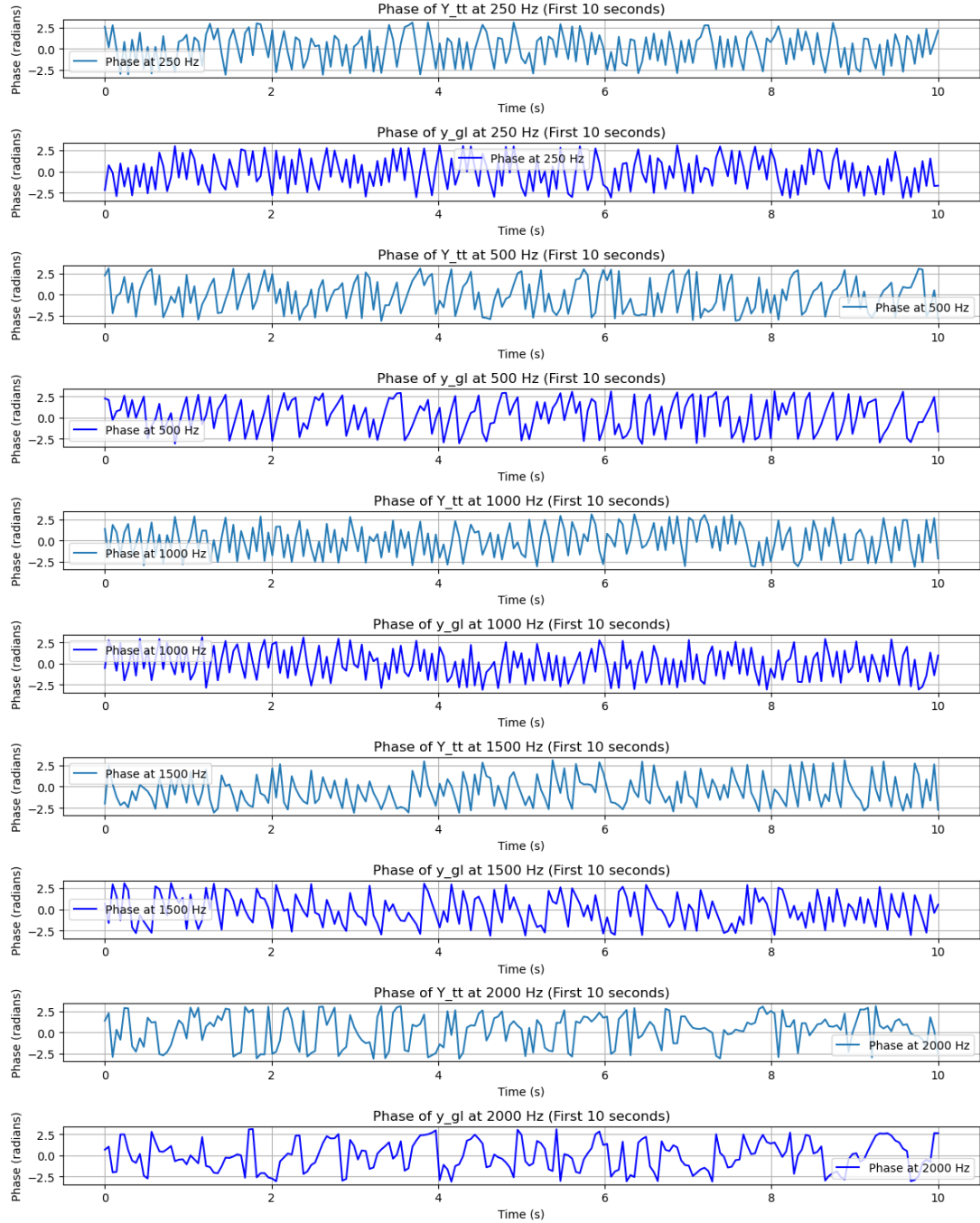


Figure 3.3: Phase representation for each frequency

## Chapter 4

# Timbre Transfer Function

Now that we have developed the whole pipeline for accomplishing timbre transfer, in this section we will have to pack everything together inside one single function, such that we can easily apply the pipeline to all the other sources and targets. We create the function named **timbre\_transfer**, which takes in the target waveform **t**, the source waveform **s**, the sampling frequency **f<sub>s</sub>**, the STFT hop size **hop\_size**, the STFT window length **win\_length**, the re-synthesis method **resynth**, and a boolean variable **plot**. It returns **y**, the audio waveform of the target with the source timbre. The function follows the steps performed in the previous points.

```
1 resynth='gf'
2 plot=False
3 def timbre_transfer(t, s, fs, hop_size=1024,
4     win_length=4096, resynth='gf', plot=False):
5     # 1. Compute magnitude STFTs of target and source
6     X_target = librosa.stft(t, n_fft=win_length,
7         hop_length=hop_size)
8     Y_target = np.abs(X_target)
9
10    X_source = librosa.stft(s, n_fft=win_length,
11        hop_length=hop_size)
12    Y_source = np.abs(X_source)
13
14    # 2. Initialize the activation matrix H
15    H0 = np.random.rand(X_source.shape[1],
16        X_target.shape[1]) # Random initialization
17
18    # 3. Initialize the template matrix W
19    epsilon = 1e-6
20    W0 = Y_source / (np.sum(Y_source, axis=0,
21        keepdims=True) + epsilon)
22
23    # 4. Normalize the STFT of the source
24    Xs = X_source / (np.sum(Y_source, axis=0,
25        keepdims=True) + epsilon)
```

```

20 # 5. Perform NMF on the target magnitude STFT
21 W, H = nmf(
22     V=Y_target,
23     cost_func='KLDiv',
24     num_iter=50,
25     init_W=W0,
26     init_H=H0,
27     fix_W=True
28 )
29
30
31 # 6. Compute the complex-domain STFT of the processed
32     track
33 Y_tt = Xs @ H
34
35 # 7. Visualize NMF matrices if plot == True
36 if plot:
37     visualize_nmf(Y_target, W, H, fs, hop_size / fs,
38         gamma=2)
39
40 # 8. Re-synthesis
41 if resynth == 'istft':
42     y = librosa.istft(Y_tt, hop_length=hop_size,
43         win_length=win_length)
44 elif resynth == 'gf':
45     y = librosa.griffinlim(np.abs(Y_tt),
46         hop_length=hop_size, win_length=win_length)
47 else:
48     raise ValueError("Invalid resynth method. Choose
49         'istft' or 'gf'.")
50
51 return y

```

# Chapter 5

## Final Results

### 5.1 Timbre transfer for all combinations

Now that we have the function `timbre_transfer` we are going to process all the combinations of sources and targets.

```
1 for target in targets:
2     for source in sources:
3         print(f"Processing Source '{source}' and Target '{target}'")
4
5         # Load the source and target audio files
6         source_path = f"{source_dir}/{source}.mp3"
7         target_path = f"{target_dir}/{target}.wav"
8
9         try:
10            source_waveform, source_sr =
11                librosa.load(source_path, sr=Fs)
12            target_waveform, target_sr =
13                librosa.load(target_path, sr=Fs)
14        except Exception as e:
15            print(f"Error loading files: {e}")
16            continue
17
18        # Perform timbre transfer
19        y = timbre_transfer(
20            t=target_waveform,
21            s=source_waveform,
22            fs=Fs,
23            hop_size=H_size,
24            win_length=N_length,
25            resynth=resynth,
26            plot=plot
27        )
28
29        # Save the resynthesized audio file
```

```

28     output_dir = "C:/Users/franc/HOMEWORK 2/results"
29     os.makedirs(output_dir, exist_ok=True)
30     method = "istft" if resynth == 'istft' else
        "griffinlim"
31     filename =
        f"{output_dir}/{source}_{target}_{method}.wav"
32     sf.write(filename, y, Fs)
33     print(f"Resynthesized audio saved at {filename}")
34
35     # Play the resynthesized audio
36     print(f"Playing result for Source '{source}' and
        Target '{target}':")
37     ipd.display(ipd.Audio(y, rate=Fs))

```

The algorithm for timbre transfer can indeed transfer the timbre of the source to the target, but its success heavily depends on several factors related to the characteristics of both the source and the target, as well as the algorithm's configuration (e.g., NMF parameters, window length, etc.). The algorithm works best when the source and target share similar spectral, harmonic, and temporal characteristics, and it struggles with pairs that have significantly different structures. The best results tend to come from simple, harmonic, and less dynamic sources and targets. So in our opinion the best results, for this set of targets, are those related to when the source is the beez or wind sound.

## 5.2 Experimenting with parameters

We are now going to experiment with some parameters in order to find the best combination of values that returns better results.

### Hop size and window length

When experimenting with the values for `H_size` (hop size) and `N_length` (window length) in the context of timbre transfer using NMF (Non-negative Matrix Factorization), the chosen parameters significantly influence the time and frequency resolutions of the Short-Time Fourier Transform (STFT), which impacts the accuracy of the timbre transfer process. Let's break down how these parameters affect the results and how to select them optimally for different source-target pairs.

- **Hop size** (`H_size`): This determines how much the window is shifted when computing the STFT. Smaller values for hop size result in higher time resolution, meaning the model will capture more precise temporal features of the signal. Larger hop sizes will give lower time resolution, which might smooth out transients but could also improve computational efficiency. Smaller hop size returns a more detailed representation of fast-changing (transient) parts of the signal, such as percussive elements. Larger hop size



is better suited for smoother signals where fine temporal details are less important.

- **Window length** (N\_length): This parameter determines the length of the window used for the STFT. A larger window length provides better frequency resolution but sacrifices time resolution, while a smaller window gives higher time resolution but worse frequency resolution. Smaller N\_length is better for transient-rich sounds (e.g., percussive or vocal elements) that change quickly in time, but sacrifices frequency precision. Larger N\_length is more suited for harmonic sounds or longer sustained notes, such as strings or wind instruments, where capturing the frequency content precisely is more important than capturing rapid changes in time.

The need to adjust these parameters stems from the nature of the source and target sounds. Different sounds have distinct time and frequency characteristics that affect the resolution required for accurate timbre transfer. We have reported the best subjective pair of values for each target- source pair.

- **Target: "Blue Christmas" with Source: "Beez Buzzing"**: Optimal values: N\_length = 4096 and H\_size = 1024.
- **Target: "Blue Christmas" with Source: "Chainsaw Sawing"**: Optimal values: N\_length = 2048 and H\_size = 1024.
- **Target: "Blue Christmas" with Source: "Wind Blowing"**: Optimal values: N\_length = 4096 and H\_size = 2048.
- **Target: "Have Yourself" with Source: "Beez Buzzing"**: Optimal values: N\_length = 4096 and H\_size = 1024,2048.
- **Target: "Have Yourself" with Source: "Chainsaw Sawing"**: Optimal values: N\_length = 4096 and H\_size = 1024.
- **Target: "Have Yourself" with Source: "Wind Blowing"**: Optimal values: N\_length = 1024 and H\_size = 512.
- **Target: "Jingle Bells Boogie" with Source: "Beez Buzzing"**: Optimal values: N\_length = 4096 and H\_size = 512,1024.
- **Target: "Jingle Bells Boogie" with Source: "Chainsaw Sawing"**: Optimal values: N\_length = 2048 and H\_size = 512.
- **Target: "Jingle Bells Boogie" with Source: "Wind Blowing"**: Optimal values: N\_length = 2048 and H\_size = 512.
- **Target: "White Christmas" with Source: "Beez Buzzing"**: Optimal values: N\_length = 2048,4096 and H\_size = 512,1024.
- **Target: "White Christmas" with Source: "Chainsaw Sawing"**: Optimal values: N\_length = 2048 and H\_size = 512.

- **Target: "White Christmas" with Source: "Wind Blowing"**: Optimal values:  $N\_length = 2048$  and  $H\_size = 512$ .

In general smaller window length (1024 or 2048) are better for targets with harmonic or melodic content (e.g., vocals, strings, and pianos) and larger window length (2048 or 4096) for complex percussive or noisy sources. On the other hand smaller hop size (256 or 512) are better for targets with rapid temporal changes (e.g., rhythmic tracks, complex sounds) and larger hop size (512 or 1024) for smoother, more sustained targets (e.g., instrumental solos, vocals).

### 5.3 NMF parameters

Now that we have experimented with the values of hop size and window length, we are going to focus on Nmf parameters such as:

- **Number of iterations**: it affects the convergence of the NMF algorithm. Too few iterations may result in an inaccurate approximation of the target's timbre, while too many can lead to overfitting.
- **Polyphony Control**: it regulates the number of templates active at a time. This is crucial for complex timbres or polyphonic audio, where more templates may be necessary.
- **Length of Continuity**: it controls the temporal coherence of the activation patterns. Too short values might introduce artifacts, while excessively long values can blur the dynamics of the audio.
- **Grid for Continuity**: it specifies how often continuity constraints are applied. A finer grid might improve consistency but increases computational cost.
- **Sparsity Control**: imposes sparsity on the activations, which is critical for clarity. Overly sparse activations might oversimplify the timbre, while insufficient sparsity could retain unwanted noise.

We obtain, with some combinations of parameters, a lower error than the initial case but it's not necessarily the best for timbre transfer. Minimizing the error excessively can lead to overfitting, where the approximation captures irrelevant details, losing the flexibility needed to adapt the timbre of the source. The objective is not just a low error but producing audio that integrates the timbre of the source while preserving the structure of the target. A lower error might sacrifice timbral realism or musical coherence. Instead of minimizing the error alone, we try to balance a sufficiently low error with a perceived high-quality output. The most relevant parameter in our opinion is the Sparsity Control (`cont_sparsen`). It directly affects how well the timbre of the source is integrated with the target audio. In our case, this parameter is crucial for ensuring that the source timbre is applied effectively without introducing significant artifacts.

Also the number of Iterations (`num_iter`) is very important. Proper convergence ensures the timbre transfer is both accurate and robust.

## 5.4 Additional processing steps to improve timbre transfer results

To improve the quality of timbre transfer, there are several signal processing and harmonic-related techniques that could be applied before, during, or after the timbre transfer process.

- **Equalizing the Frequency Content of Source and Target (Pre-processing):** Before applying timbre transfer, we could preprocess both the source and target signals to ensure they are in similar frequency ranges. Sometimes, sources and targets can have different spectral content that makes the timbre transfer less effective. A bandpass filter could be applied to both source and target signals to remove low-end or high-end frequencies that might not be relevant to the timbre. This would ensure that both signals share similar frequency distributions, potentially improving the timbre matching.
- **Pitch Shifting (Pre-processing):** In some cases, the pitch of the source and target may differ, affecting their harmonic content and overall timbre. We could use pitch shifting algorithms to shift the pitch of the source to match the pitch of the target sound. This would involve both shifting the fundamental frequency and adjusting the harmonics accordingly. This would ensure that the harmonic series of the source aligns more closely with that of the target, improving the perceived similarity between the two timbres.
- **Spectral Smoothing:** In certain cases, the NMF decomposition might create very sharp transitions in the spectral features, resulting in an unnatural sound. Spectral smoothing could help make the timbre more cohesive and natural. Applying techniques such as moving average filters or Gaussian smoothing to reduce high-frequency fluctuations in the spectral content. This could be applied either before or after the timbre transfer process. Smoothing would create a more continuous and smooth spectral representation, improving the perceived naturalness of the transferred timbre.
- **Reverb and Spatial Effects (Post-Processing):** Often, timbre is not only influenced by spectral content but also by the spatial characteristics of the sound (e.g., room acoustics or the reverb characteristics of the target environment). We could apply reverb matching techniques to match the spatial effects (e.g., early reflections, reverb tail) of the source to those of the target. This might involve using reverberation models or even reverb

convolution with the target environment’s impulse response. The timbre transfer would sound more natural and realistic in the target context, particularly for sounds that have specific spatial or environmental qualities.

### 5.4.1 Checking timbre similarities

To determine the similarity of timbres between two songs, a common approach is to compare their spectral characteristics. Among various methods, the most efficient and computationally fast technique is the Mel-frequency Cepstral Coefficients (MFCC) Comparison. MFCCs are widely used in speech and audio analysis as they effectively represent the timbral features of a sound by capturing both harmonic and non-harmonic content. By extracting MFCCs from both audio files and calculating a similarity score (such as Euclidean distance or cosine similarity) we can quantify their timbral similarity. This method is particularly efficient because MFCCs reduce the dimensionality of the data while preserving essential spectral information. Additionally, compared to other techniques like Kullback-Leibler Divergence, MFCC comparison is less computationally expensive, making it ideal for quick and effective timbre analysis. A smaller distance between MFCCs indicates greater timbral similarity. Here an implementation that confront the timbre of the sources with the timbre of the targets:

```

1 def compare_audio_mfcc(path1, path2):
2     # Load the two audio files
3     y1, sr1 = librosa.load(path1, sr=None)
4     y2, sr2 = librosa.load(path2, sr=None)
5     # Extract MFCCs (Mel-frequency Cepstral Coefficients)
6     # from both files
7     mfcc1 = librosa.feature.mfcc(y=y1, sr=sr1, n_mfcc=13)
8     mfcc2 = librosa.feature.mfcc(y=y2, sr=sr2, n_mfcc=13)
9     # Compute the mean of the MFCCs
10    mfcc1_mean = np.mean(mfcc1, axis=1)
11    mfcc2_mean = np.mean(mfcc2, axis=1)
12    # Compute the Euclidean distance between the mean
13    # MFCCs of both files
14    distance = np.linalg.norm(mfcc1_mean - mfcc2_mean)
15    return distance

```

The function effectively analyzes the timbral similarity between sources and targets files using MFCCs. It computes the Euclidean distance between their mean MFCCs, providing a numerical measure of timbral dissimilarity. Initial results suggest that smaller distances correlate with higher timbral similarity.

Potential areas for refinement include incorporating additional MFCC statistics (e.g., standard deviation) and experimenting with alternative similarity metrics. Pre-processing steps such as normalization and silence trimming could enhance robustness. The couples with the lower timbral distances result being:

- Source **Beez Buzzing** and Target **Blue Christmas** with a distance of 96.33

- Source **Wind Blowing** and Target **Have Yourself** with a distance of 92.68
- Source **Wind Blowing** and Target **Jingle Bells Boogie** with a distance of 72.68

Then we proceed confronting the timbre of the targets with the results of timbre-transfer application with the relative target and report the lower distance for each one:

- Target **Blue Christmas** and timbre transfer with source **Chainsaw Sawing** 68.4
- Target **White Christmas** and timbre transfer with source **Wind Blowing** 94.50
- Target **Have Yourself** and timbre transfer with source **Bees Buzzing** 69.54
- Target **Jingle Bells Boogie** and timbre transfer with source **Wind Blowing** 99.25

Finally we confront the timbre of the sources with the results of timbre-transfer application:

- Source **Bees Buzzing** and timbre transfer with source **Chainsaw Sawing** 68.4
- Source **Chainsaw Sawing** and timbre transfer with source **Wind Blowing** 94.50
- Source **Wind Blowing** and timbre transfer with source **Bees Buzzing** 69.54